

Desarrollo asistido por Inteligencia Artificial de un resolutor de sistemas de ecuaciones complejas para análisis de circuitos

Orlando Contreras Reyes, Iker Emiliano García Barrera,
Dr. Luís Alejandro Flores Oropeza, Dr. Alejandro Román Loera

Universidad Autónoma de Aguascalientes a1348390@edu.uaa.mx

Introducción

El análisis de circuitos (mallas o nodos) exige resolver sistemas de ecuaciones con números complejos. Las soluciones habituales presentan limitaciones:

- **Regla de Cramer:** método poco eficiente.
- **Calculadoras físicas y simuladores** (p. ej. HP-48G, Droid 48): ingreso de datos poco eficiente y resultados solo en forma rectangular o solo polar.
- **Deshonestidad académica:** uso de herramientas sin comprender el proceso.

Objetivo. Crear una aplicación de escritorio, portátil y gratuita que resolviera sistemas de ecuaciones complejas y muestre los resultados en ambas formas, desarrollada junto con IA.

Métodos: herramientas

- **Lenguaje y entorno:** Python 3.10+, Visual Studio Code.
- NumPy: álgebra lineal compleja.
- CustomTkinter y Tkinter: interfaz gráfica.
- Pillow, OS y JSON: recursos gráficos, archivos e historial de matrices.
- **IA:** ChatGPT (OpenAI) para código base, refactorización y diseño de la GUI.
- PyInstaller: ejecutable único (-onefile, -noconsole) para Windows y GNU/Linux.

Filosofía

Open Source y *Vibe Coding*: el proyecto lo realizaron alumnos con bases de programación en C para microcontroladores. Con buenas bases, el ingeniero puede construir sus propias herramientas. Se publica bajo licencia MIT para que otros estudiantes aprendan, contribuyan y la reutilicen.

Métodos: arquitectura y algoritmo



Fig. 1: Arquitectura modular.

Un sistema de $n \times n$ se plantea como $[Z][I] = [V] \Rightarrow [I] = [Z]^{-1}[V]$ y se resuelve con `numpy.linalg.solve`. Los resultados se convierten a forma polar:

$$r = \sqrt{a^2 + b^2}, \quad \theta = \arctan\left(\frac{b}{a}\right)$$

Entrada en forma rectangular ($a + jb$) o polar ($r\angle\theta^\circ$).

Flujo de trabajo con IA

1. Maquetas iniciales de ventanas y celdas matriciales editables.
2. Solución de errores con arreglos complejos en tiempo real.
3. Refactorización: separar cálculo y renderizado.
4. **Validación humana:** cada fragmento fue revisado, probado y contrastado con ejercicios resueltos a mano.

Discusión

- Útil en Análisis de Circuitos, Electrónica, Señales y Sistemas y Métodos Matemáticos.
- Historial de operaciones y guardado/carga de matrices.
- Interfaz reorganizada para ser clara a usuarios sin experiencia.
- Tkinter limita los temas visuales; se personalizó un archivo .json de estilos.

Resultados



Fig. 2: Solución de un sistema complejo en forma rectangular.



Fig. 3: Solución de un sistema complejo en forma polar.

Conclusiones

1. La IA es un apoyo, no un sustituto del conocimiento del programador: reduce el tiempo de desarrollo, pero exige entender y validar el código.
2. Con bases de C para microcontroladores se logró una aplicación funcional con entrada y conversión rectangular/polar.
3. El ejecutable facilita su distribución sin configurar el entorno.
4. Licencia MIT: base abierta para mejoras y herramientas similares.

Referencias

- [1] H. Anton y C. Rorres, *Álgebra lineal con aplicaciones*. Wiley, 2013.
- [2] D. C. Lay, *Álgebra lineal y sus aplicaciones*, 4.ª ed. Pearson, 2012.
- [3] NumPy Developers, *numpy.linalg.solve*. NumPy Documentation, 2026.
- [4] Python Software Foundation, *Built-in Types* y *tkinter*. Python Documentation, 2026.
- [5] D. Cortesi et al., *PyInstaller Manual*, 2026.
- [6] OpenAI, *ChatGPT*, 2026.

Contacto y código

a1348390@edu.uaa.mx
<https://github.com/DasReyxr/Py-ComplexCalc>